Oracle

When a `ResultSet` is being traversed, the client-side cursor fetches data in chunks (the fetch size attribute controlling the number of records in a chunk).

After each new batch retrieval, the total number of records is checked against the `maxRows` upper bound, and if the threshold is reached, the driver closes the networking stream.

The `maxRows` upper bound can, therefore, prevent the database and the client-side driver from wasting resources on fetching records the client does not even need. However, if the `maxRows` value is small, the Optimizer will not use indexes if the size of the scanned data set is rather large.

SQL Server

When the `Statement.setMaxRows(int maxRows)`[a] method is called, the driver calls the `SET ROWCOUNT` SQL command:

```
SET ROWCOUNT N
```

Unlike the `TOP` or `FETCH` SQL directives, the `ROWCOUNT` command is taken into consideration only during the execution phase, and it does not influence the plan generation. Because of this, the execution plan might not be optimized for the given result set size, so a table scan might be chosen over an index.

The SQL Server documentation[b] recommends using the SQL directives over the `SET ROWCOUNT` command.

[a] https://msdn.microsoft.com/en-us/library/ms378838%28v=sql.110%29.aspx

PostgreSQL

The JDBC driver takes the `maxRows` statement attribute and sends it along with the query being executed. With this info, the Optimizer can choose an execution plan that is tailored for the given result set size, and it might even avoid some expensive operations like sorting the whole projection. The Extractor can also close the database cursor right after it fetched the desired number of records, therefore sparing both database and networking resources.

> **MySQL**
>
> The `maxRows` attribute is not sent to the database server, so neither the Optimizer nor the Extractor can benefit from this hint. While the JDBC driver would normally fetch all rows, by placing an upper bound on the result set size, the client-side can spare some networking overhead.

5.5.1.3 Less is more

The following test is going to demonstrate the performance improvement of limiting the result set size. The test data set consists of 100 000 *post* and 1 000 000 *comment* entries. In the first round, the entire result set is being fetched, and the response time is going to be proportional to the projection size. By limiting the result set to 100 records, either by using SQL or the JDBC `maxRows` setting, the response time is going to drop significantly.

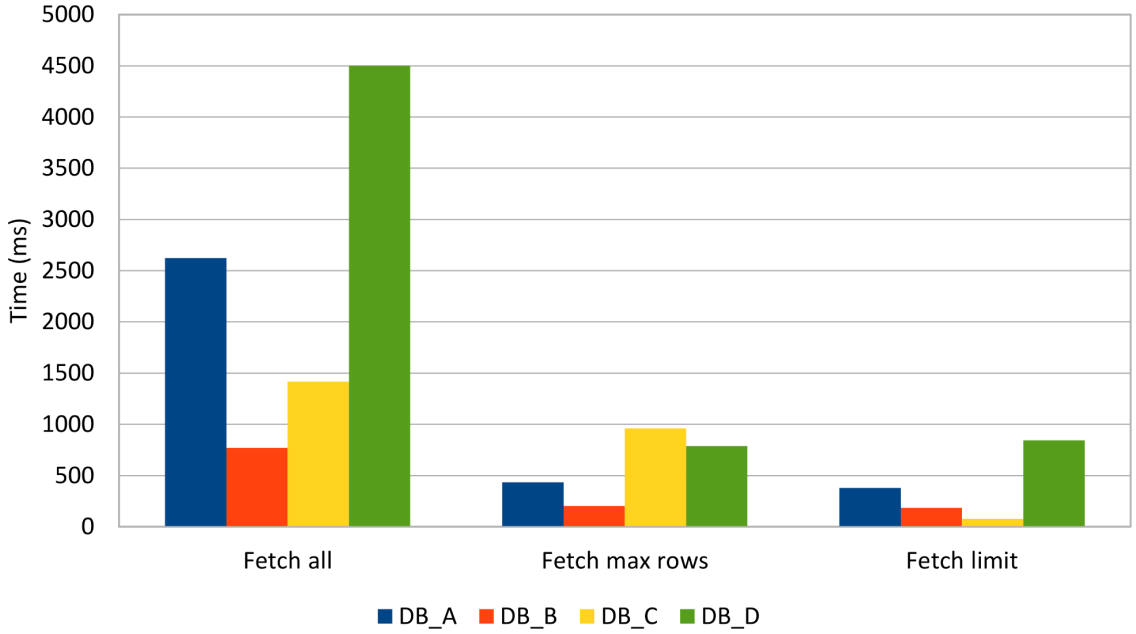

Figure 5.3: ResultSet size

The test results confirm the previous assumptions and the SQL level restriction proves to be the optimal strategy for limiting a result set. The `maxRows` driver implementation yields a surprisingly good result, especially when taking into consideration the JDBC specification on dropping extra records. Fetching a large result set puts much pressure on database resources, which does not only affect the current processing unit of work. Other concurrent transactions can also exhibit longer processing times, as a consequence of database resources shortage.

5.5.2 Too many columns

Not only fetching too many rows can cause performance issues but even extracting too many columns can increase the result set processing response time. The next test case is going to select 100 *posts* with *details* and their associated 1000 *comments*, using one of following two statements:

```
SELECT *
FROM post_comment pc
INNER JOIN post p ON p.id = pc.post_id
INNER JOIN post_details pd ON p.id = pd.id

SELECT pc.version
FROM post_comment pc
INNER JOIN post p ON p.id = pc.post_id
INNER JOIN post_details pd ON p.id = pd.id
```

The following graph depicts the execution times of fetching all columns, as opposed to extracting only a subset of the whole column projection.

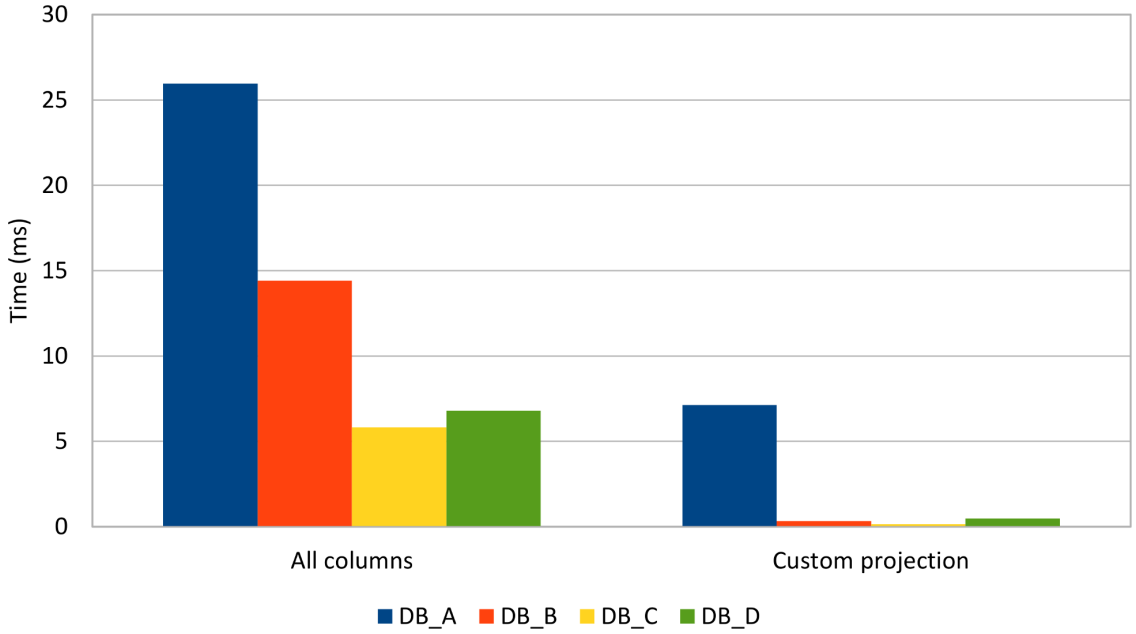

Figure 5.4: ResultSet projection size

This situation is more prevalent among ORM tools, as for populating entities entirely, all columns are needed to be selected. This might pass unnoticed when selecting just a few entities, but, for large result sets, this can turn into a noticeable performance issue.

If a business case requires only a subset of all entity attributes, fetching extra columns will become a waste of database and application resources (CPU, Memory, I/O, Networking).

6. Transactions

A database system must allow concurrent access to the underlying data. However, shared data means that read and write operations must be synchronized to ensure that data integrity is not compromised.

To control concurrent modifications, the Java programming language defines the `synchronized` keyword for two purposes:

- It can restrict access to a shared `Object` (to preserve invariants), so only a `Thread` can execute a routine at any given time.
- It propagates changes from the current `Thread` local memory to the global memory that is available to all running threads of executions.

This behavior is typical for other concurrent programming environments and database systems are no different. In a relational database, the mechanism for ensuring data integrity is implemented on top of transactions.

A transaction is a collection of read and write operations that can either succeed or fail together, as a unit. All database statements must execute within a transactional context, even when the database client does not explicitly define its boundaries.

In 1981, Jim Gray first defined the properties of a database transaction in his famous paper: The transaction concept: virtues and limitations[1]. Both this paper and the first versions of the SQL standard (SQL-86 and SQL-89) only used three properties for defining a database transaction: *Atomicity*, *Consistency*, and *Durability*.

Along with other relation database topics, the transaction research has continued ever since, and so the SQL-92 version introduced the concept of *Isolation Levels*. These four properties have been assembled in the well-known *ACID* (*Atomicity*, *Consistency*, *Isolation*, *and Durability*) acronym that soon became synonym with relation database transactions.

Knowing how database transactions work is very important for two main reasons:

- effective data access (data integrity should not be compromised when aiming for high-performance)
- efficient data access (reducing contention can minimize transaction response time which, in turn, increases throughput).

The next sections detail each transaction property in relation to high-performance data processing.

[1] http://research.microsoft.com/en-us/um/people/gray/papers/theTransactionConcept.pdf

6.1 Atomicity

Atomicity is the property of grouping multiple operations into an all-or-nothing unit of work, which can succeed only if all individual operations succeed. For this reason, the database must be able to roll back all actions associated with every executed statement.

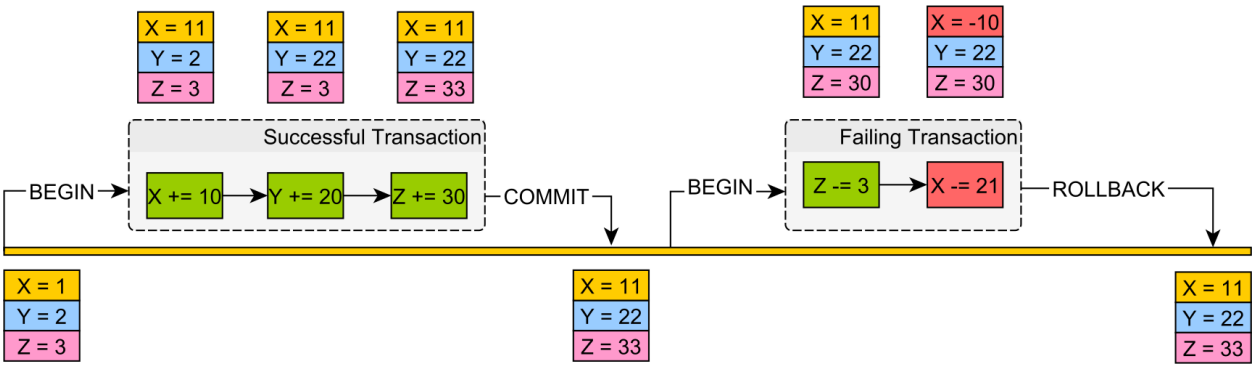

Figure 6.1: Atomic units of work

Write-write conflicts

Ideally, every transaction would have a completely isolated branch which could be easily discarded in case of a rollback. This scenario would be similar to how a *Version Control System* (e.g. *git*) implements branching. In case of conflicts, the Virtual Control System aborts the commit operation, and the client has to manually resolve the conflict. Unlike VCS tools, the relational database engine must manage conflicts without any human intervention.

For this reason, the database prevents write-write conflict situations, and only one transaction can write a record at any given time.

All statements are executed against the actual data structures (tables, indexes, in-memory buffers), only to be materialized at commit time. In case of rollback, the database must revert any pending changed datum to its previous state.

Oracle

The *undo tablespace*[a] stores the previous data versions in *undo segments*. Upon rolling back, the database engine searches the associated *undo segments* that can recreate the *before image* of every datum that was changed by the currently running transaction.

[a] https://docs.oracle.com/database/121/ADMIN/undo.htm#ADMIN11460